Lab 11: Automated Unit Testing and Coverage Analysis

E-Commerce Order and Inventory Fulfillment System

1. Objective

In this laboratory, you act as a Software Quality Assurance (QA) and Test Engineer. You are provided with a complete, backend-only E-Commerce Order and Inventory Fulfillment System that simulates the core workflows of platforms such as Amazon or Flipkart: multi-warehouse stock reservation, promotional discount engines, order state machine progression, logistics courier allocation, return policies and financial settlements. Your task is to:

- Analyse and study the modular architecture and business logic of each module.
- Develop automated test suites using pytest without modifying any production source code.
- Achieve at least 90% line coverage and at least 90% branch coverage using pytest-cov / coverage.

Source of Code. The ecommerce_fulfillment code used in this lab is the code attached in the Google Classroom assignment. You must use it as it is, without any modification.

2. Environment Setup

From the ecommerce_fulfillment root directory, install the test dependencies and verify the tools:

```
pip install -r requirements.txt

pytest --version
coverage --version
```

3. Codebase Architecture and Study Guide

All source code is located under src/ecommerce_fulfillment/. Study the modules in the following sequence before writing any tests.

Step	Module	File	Core concepts to inspect
1	Exceptions	exceptions.py	Custom error hierarchy derived from EcommerceError. Every domain error (e.g. InsufficientStockError, InvalidOrderStateTransitionError, ReturnPolicyViolationError) raises a specific exception.
2	Models and Enums	models.py	Dataclasses (User, Address, Product, WarehouseInventory, Order, OrderItem, PricingBreakdown, Shipment, Transaction) and enums (MembershipTier, ProductCategory, OrderStatus, PaymentMethod, CourierPartner).
3	Utilities	utils.py	Clock (injectable for time control and return-window testing), IdGenerator (deterministic sequential IDs), validation patterns for email, phone and 6-digit postal pincodes.
4	User Management	user.py	Customer and seller registration, delivery address management, membership tier upgrades (STANDARD, PRIME, VIP), and prepaid wallet credits and debits.
5	Inventory Management	inventory.py	Product cataloguing, multi-warehouse stock replenishment, stock reservation locking on order creation, stock release on cancellation, and permanent stock commitment on dispatch.

Step	Module	File	Core concepts to inspect
6	Pricing Engine	pricing.py	Category-specific GST (Groceries 5%, Books 0%, Apparel 12%, Electronics 18%), coupon validation (expiry date, minimum order value, maximum usages), and tiered free delivery (Prime/VIP, or orders of ₹500 and above).
7	Order Lifecycle	order.py	Strict 8-stage state machine: CREATED, CONFIRMED, PACKED, SHIPPED, OUT_FOR_DELIVERY, DELIVERED, CANCELLED, RETURNED. Rollback of reserved inventory if order validation fails.
8	Logistics and Shipping	shipping.py	Courier dispatch (BlueDart for parcels above 10 kg, Ekart for metro pincodes, Delhivery for other regional zones), tracking number assignment, checkpoint updates and delivery confirmation.
9	Payment Management	payment.py	PaymentGateway interface, MockPaymentGateway with failure triggers, prepaid wallet debits, Cash-on-Delivery (COD) logging, external card/UPI settlement and financial ledger tracking.
10	Returns and Cancellations	cancellation.py	Restocking fee assessment (₹50 if cancelled after dispatch), 7-day return window verification, warehouse restock on return, and automated refunds to wallet or original gateway.
11	Notifications	notification.py	Mock SMS and Email channels, order confirmation receipts, dispatch SMS with tracking numbers, and message history auditing.
12	Analytics	analytics.py	Gross Merchandise Value (GMV), category sales breakdown, seller net payout after 10% platform commission, and return rate percentages.
13	System Facade	system.py	EcommerceFulfillmentSystem orchestrating end-to-end shopping, fulfillment, delivery and testing workflows.

4. Writing Unit Tests

4.1 Required Test Layout

Create a tests/ directory at the project root with the following structure:

```
ecommerce_fulfillment/
├── src/
│   └── ecommerce_fulfillment/
├── tests/
│   ├── conftest.py
│   ├── test_user.py
│   ├── test_inventory.py
│   ├── test_pricing.py
│   ├── test_order.py
│   ├── test_shipping.py
│   ├── test_payment.py
│   ├── test_cancellation.py
│   ├── test_notification.py
│   ├── test_analytics.py
│   └── test_system.py
└── requirements.txt
```

4.2 Testing Strategies

- Happy paths and error branches. Test normal order creation, checkout and delivery, as well as edge cases such as ordering more than available stock, invalid pincodes, negative wallet balances and illegal order state transitions.
- Deterministic time injection. Use Clock and clock.advance_days(...) to test promotional coupon expiry, and return-window eligibility (within 7 days versus a return attempt on day 8).

- Mock interfaces. Use MockPaymentGateway with set_failure_trigger(...) to test payment failure branches. Use MockSmsService and MockEmailService with should_succeed=False to exercise notification failure handling.
- Boundary values. Test free shipping thresholds (₹499 versus ₹500 for Standard members, and free shipping for Prime members on any amount), coupon discount caps, and the package weight threshold (above 10 kg).
- Fixtures. Place shared fixtures (clock, customer, seller, warehouse stock, mock gateway) in conftest.py.

5. Running Tests and Generating Coverage Reports

Run the following commands from the ecommerce_fulfillment directory.

5.1 Standard test run

```
pytest
```

5.2 Line and branch coverage in the terminal

```
pytest --cov=src/ecommerce_fulfillment --cov-branch --cov-report=term-missing
```

5.3 Interactive HTML coverage report

```
pytest --cov=src/ecommerce_fulfillment --cov-branch --cov-report=html
```

Open htmlcov/index.html in a browser to inspect lines and branches that were missed. Missed lines are highlighted in red and partially covered branches in yellow.

6. Submission Requirements

Submit a single PDF report. The report must contain:

- A test case table listing, for each test module, the test ID, the module and behaviour under test, the input or setup, the expected result and the actual result (pass or fail).
- Screenshots of the terminal output from the coverage command in Section 5.2, showing the line and branch coverage percentages (must be > 90%) and the missed-lines column.
- A screenshot of the terminal output from the standard pytest run in Section 5.1, showing the pass count.

Name the report using your student ID, for example 202301234_Lab11_Report.pdf.